Universidade Estadual do Tocantins (UNITINS)
Sistemas de Informação — Sistemas Distribuídos — 7º período, 2026/2

Processamento distribuído de grande volume de dados com arquitetura mestre/workers

Pré-projeto

Integrantes
Alêkson Castro
Italo Beckman
Matheus Nardi

Professor: Prof. Mailson Oliveira
Palmas – TO, 22/10/2026

Problema e cenário

Um dos quatro cenários propostos: processamento de grande volume de dados

> Processar um conjunto de dados **grande demais** para uma máquina só em tempo razoável, **dividindo o trabalho** entre várias máquinas que processam partes ao mesmo tempo e depois **combinam** os resultados.

- **Objetivo:** construir um sistema mestre/workers e medir *speedup*, balanceamento de carga e tolerância a falhas
- **Caráter exploratório:** o dataset e a pergunta que o sistema responde ainda estão em análise
- **Técnica primeiro, negócio depois:** o núcleo não depende do tema dos dados

Por que é um problema distribuído?

Uma máquina só tem limites; distribuir contorna esses limites, mas traz novos desafios

CPU

Poucos núcleos: o tempo cresce junto com o volume de dados

Memória

A partir de certo ponto, os dados nem cabem em uma máquina

Disco

A velocidade de leitura também limita o processamento

Heterogeneidade

Máquinas com desempenhos diferentes

Falhas

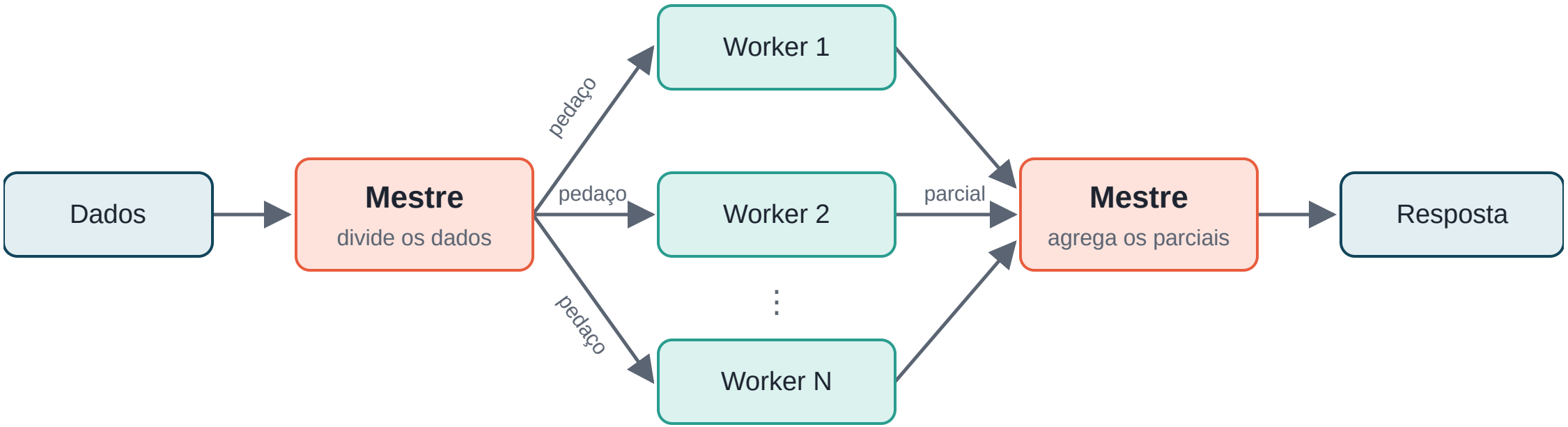
Um nó pode cair no meio da execução

Comunicação

Mandar dados pela rede tem custo

Arquitetura mestre/workers

Modelo MapReduce: dividir → processar em paralelo → combinar



Os workers devolvem **resultados parciais pequenos** (uma soma, uma contagem), nunca os dados brutos. Processamento em **lote**: começo, meio e fim.

Divisão estática × fila dinâmica

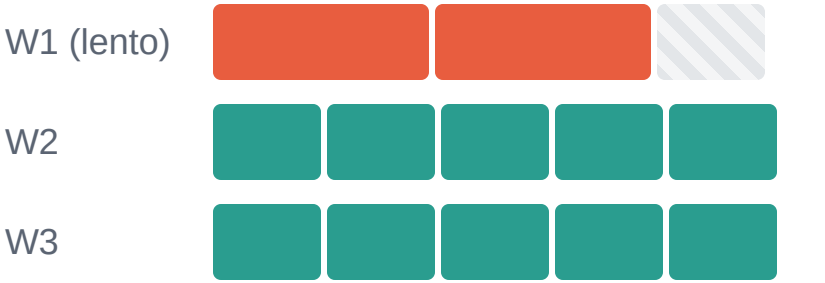
Como entregar os pedaços aos workers? O grupo vai comparar as duas formas

Estática: N partes iguais



Todos esperam o mais lento: **8 unidades de tempo**

Dinâmica: *self-scheduling*



Quem termina pede o próximo pedaço: **5 unidades**

worker normal · worker lento (2× mais lento) · tempo ocioso

Exemplo ilustrativo com 12 pedaços

Exploração dos conjuntos de dados

O dataset precisa se encaixar no paralelismo de dados

Registros independentes

Cada linha é processada sem olhar as outras

Parcial combinável

Soma, contagem, média (soma + contagem); mediana não

Volume

~100 MB para validar; 1 a 5 GB para os experimentos

Aberto

Download público, script de obtenção versionado

Candidato	Pergunta possível	Processamento
Microdados do ENEM (INEP)	Média de notas por estado, escola ou renda	Agregação por grupo
NYC Taxi Trip Records	Corridas e receita por hora ou região	Agregação e filtragem
Dumps da Wikipédia	Palavras mais frequentes	Contagem
Sintéticos (*seed* fixa)	Qualquer, com volume controlado	Qualquer

Tecnologias

Comunicação com MPI; montagem do cluster automatizada



Open MPI

MPI

Padrão de troca de mensagens da computação em cluster. `scatter` espalha os pedaços, `gather` recolhe os resultados, ponto a ponto para a fila dinâmica.





Vagrant VirtualBox

Vagrant + VirtualBox

Cria as VMs de validação no VirtualBox a partir de um `Vagrantfile` versionado: rede interna, IPs fixos e SSH.



Ansible

Ansible

Configura os nós: MPI, chaves SSH, hostfile e cópia do programa. O mesmo playbook serve às VMs e aos notebooks.

Alternativas consideradas: sockets, gRPC e RabbitMQ. Para comparação: Dask, Ray ou Spark.

Ambiente de execução

VMs na mesma máquina dividem a CPU e distorcem o speedup: um ambiente por etapa




Docker Open MPI

Desenvolvimento

Máquina local ou Docker

`mpirun -n 4` já roda 4 processos





Vagrant VirtualBox Ubuntu

Validação

VMs Ubuntu Server (1 mestre + 2 a 3 workers)

Rede e SSH reais; simular falhas desligando uma VM




Ubuntu Ansible

Medição

Notebooks físicos do grupo em rede

Speedup real, sem disputa de CPU

Ubuntu Server em vez de distro de cluster (PelicanHPC): a distro só automatiza a configuração do MPI; fazendo nós mesmos, temos mais controle e a montagem vira aprendizado.

Como vamos saber se funcionou?

Quatro perguntas, cada uma com uma métrica

Ficou mais rápido?

Speedup = tempo com 1 worker ÷ tempo com N

Ex.: 100 s ÷ 30 s ≈ **3,3×** mais rápido com 4 workers

Cada worker a mais valeu a pena?

Eficiência = speedup ÷ nº de workers

Ex.: 3,3 ÷ 4 ≈ **83%** do ganho ideal

Alguém ficou parado esperando?

Distribuição da carga

Quantos pedaços cada worker processou e quanto tempo ficou ocioso

A resposta está certa?

Corretude

Mesmo resultado da versão sequencial, mesmo quando um worker cai

Por que não 4× com 4 workers? Ler os dados, juntar os resultados e a rede não ficam mais rápidos com mais workers. Essa parte limita o ganho (Lei de Amdahl).

Números ilustrativos; os reais virão dos experimentos.

Experimentos planejados

Todos rodados nos notebooks físicos

1

Escalabilidade

Tempo e speedup com 1, 2 e 4 workers. Efeito do tamanho do pedaço: quando a comunicação passa a dominar?

2

Estática × dinâmica

Cluster heterogêneo (máquinas diferentes ou atraso artificial): tempo total, pedaços por worker e ociosidade.

3

Tolerância a falhas

Derrubar um worker no meio da execução e verificar se o mestre reatribui o pedaço dele.

Cronograma

Entregas semanais às quintas; semanas 1 e 2 em paralelo para ganhar folga

Etapa	Entrega	Data
Pré-projeto	**Apresentação da proposta**	**22/10/2026**
Semana 1	Análise dos datasets; escolha do dataset e da pergunta	29/10/2026
Semana 2	Cluster de validação (VMs, Vagrant e Ansible)	05/11/2026
Semana 3	Versão sequencial de referência + primeira versão distribuída (estática)	12/11/2026
Semana 4	Fila dinâmica + experimento de escalabilidade	19/11/2026
Semana 5	Tolerância a falhas + experimento estático × dinâmico	26/11/2026
Final	Consolidação dos resultados e apresentação final	03/12/2026

Divisão de papéis

Papéis fixos por frente: sem tempo de adaptação a cada semana

Dados e documentação

Preparo dos datasets, consolidação dos relatórios, slides e registro das decisões

Desenvolvimento

Código MPI (mestre, workers, fila dinâmica, falhas) e versão sequencial de referência

Infraestrutura e experimentos

Vagrant, Ansible e cluster; execução dos experimentos e coleta de métricas

O papel indica **quem responde** pela frente, não um limite: todos participam um pouco de tudo. Trabalho versionado em `github.com/Matheus-Nardi/projeto-sd`.

Riscos e desafios

E como pretendemos lidar com cada um

Risco	Mitigação
Máquinas heterogêneas	Fila dinâmica de tarefas
Falha de workers	Mestre registra os pedaços de cada worker e reatribui os de quem parar de responder
Custo de comunicação	Medir o efeito do tamanho do pedaço
Configuração do ambiente	Automação com Vagrant + Ansible; começar em uma máquina só
Prazo curto	Priorizar speedup e estática × dinâmica; simplificar a tolerância a falhas se apertar

Obrigado!

Dúvidas e sugestões?

github.com/Matheus-Nardi/projeto-sd